Security Code Review - Task 1

Application: Quotient Social Networking Platform **Review Date:** April 1, 2026 **Reviewer:** Security Engineering Team **Scope:** Input Validation, Output Encoding, Logging & Monitoring

---

Executive Summary

Category	Status	Risk Level	Findings
Input Validation	Implemented	Low	Minor improvements recommended
Output Encoding	Partial	Medium	Vue.js auto-escaping helps, but vanilla JS needs attention
Logging & Monitoring	Partial	Medium	Basic logging exists, needs enhancement

---

A) Input Validation

A.i) Entry Points Identified

#	Entry Point	Type	Method	Endpoint/Location
1	Login Form	Form	POST	`/login`
2	Registration Form	Form	POST	`/register`
3	Create Community Form	Form	POST	`/create-community`
4	Create Post Form	Form	POST	`/create-post`
5	Join Community	API	POST	`/join-community`
6	Leave Community	API	POST	`/leave-community`
7	Add Comment	API	POST	`/api/post/:id/comment`
8	Add Reaction	API	POST	`/api/post/:id/react`
9	Search Posts	Query Param	GET	`/api/search-posts?query=`
10	Get Post by ID	URL Param	GET	`/api/post/:id`
11	Search Bar (Community)	Query Input	Client-side	`handleSearch()`
12	URL Parameters	URL Param	GET	`?id=`, `?search=`

Headers Analyzed:

- `Content-Type` - Parsed by express.json() and express.urlencoded()
- No custom headers processed from user input

File Uploads: None identified in the application

A.ii) Server-Side Validation Analysis

Entry Point 1: Login (`/login`)

```
app.post("/login", loginLimiter, validateLogin, handleValidationErrors, async (req,
res) => { ... });

const validateLogin = [
    body("username").trim().notEmpty().withMessage("Username is required"),
    body("password").notEmpty().withMessage("Password is required")
];
```

Check	Status	Details
Type Validation	complete	express-validator handles type checking
Length Validation	not implemented	No max length specified for username/password
Format Validation	not implemented	No pattern validation for username
Allowlist Used	N/A	Free-text input
Sanitization	complete	Global `sanitizeInput` middleware applies `filterXSS`
Parameterized Query	complete	Mongoose ORM used (`User.findOne()`)

Recommendation: Add length limits: `.isLength({ min: 1, max: 50 })`

Entry Point 2: Registration (`/register`)

```
app.post("/register", registerLimiter, validateRegistration, handleValidationErrors,
async (req, res) => { ... });

const validateRegistration = [
    body("name").trim().notEmpty().withMessage("Name is required").isLength({ max:
100 }).withMessage("Name too long"),
    body("email").trim().notEmpty().withMessage("Email is
required").isEmail().normalizeEmail().withMessage("Valid email required"),
    body("username").trim().notEmpty().withMessage("Username is
required").isLength({ min: 3, max: 30 }).withMessage("Username must be 3-30
characters").matches(/^[a-zA-Z0-9_]+$/).withMessage("Username can only contain
letters, numbers, and underscores"),
    body("password").notEmpty().withMessage("Password is required").isLength({ min:
8, max: 128 }).withMessage("Password must be 8-128 characters")
];
```

Check	Status	Details
Type Validation	complete	express-validator handles type checking
Length Validation	complete	Name (100), Username (3-30), Password (8-128)

Format Validation	complete	Email format, Username pattern (`^[a-zA-Z0-9_]+$`)
Allowlist Used	complete	Username uses regex allowlist pattern
Sanitization	complete	`filterXSS` + `normalizeEmail()`
Parameterized Query	complete	Mongoose ORM used

Status: complete **WELL IMPLEMENTED**

Entry Point 3: Create Community (`/create-community`)

```
app.post("/create-community", validateCommunity, handleValidationErrors, async (req,
res) => { ... });

const validateCommunity = [
    body("community").trim().notEmpty().withMessage("Community name is
required").isLength({ max: 100 }).withMessage("Community name too long"),
    body("age").isInt({ min: 13, max: 21 }).withMessage("Age must be between 13 and
21"),
    body("description").trim().notEmpty().withMessage("Description is
required").isLength({ max: 1000 }).withMessage("Description too long")
];
```

Check	Status	Details
Type Validation	complete	`isInt()` for age, string for others
Length Validation	complete	Community (100), Description (1000)
Format Validation	complete	Integer range for age (13-21)
Allowlist Used	not implemented	No pattern validation for community name
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	complete	Mongoose ORM used

Recommendation: Add pattern validation for community name similar to username.

Entry Point 4: Create Post (`/create-post`)

```
app.post("/create-post", validatePost, handleValidationErrors, async (req, res) => {
... });

const validatePost = [
    body("title").trim().notEmpty().withMessage("Title is required").isLength({ min:
3, max: 100 }).withMessage("Title must be 3-100 characters"),
    body("brief").trim().notEmpty().withMessage("Brief is required").isLength({ min:
10, max: 500 }).withMessage("Brief must be 10-500 characters"),
```

```
    body("community").trim().notEmpty().withMessage("Community is required")
];
```

Check	Status	Details
Type Validation	complete	String validation via express-validator
Length Validation	complete	Title (3-100), Brief (10-500)
Format Validation	not implemented	No pattern validation
Allowlist Used	not implemented	Community should be validated against existing communities
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	complete	Mongoose ORM used

Recommendation: Add community existence validation before post creation.

Entry Point 5: Add Comment (`/api/post/:id/comment`)

```
app.post('/api/post/:id/comment', validateId, handleValidationErrors,
validateComment, handleValidationErrors, async (req, res) => { ... });

const validateComment = [
    body("content").trim().notEmpty().withMessage("Comment content is
required").isLength({ max: 1000 }).withMessage("Comment too long")
];

const validateId = [
    param("id").isMongoId().withMessage("Invalid post ID")
];
```

Check	Status	Details
Type Validation	complete	MongoDB ID format validated
Length Validation	complete	Content max 1000 characters
Format Validation	complete	`isMongoId()` validates ObjectId format
Allowlist Used	N/A	Free-text comment
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	complete	Mongoose ORM used

Status: complete **WELL IMPLEMENTED**

Entry Point 6: Add Reaction (`/api/post/:id/react`)

```
app.post('/api/post/:id/react', validateId, handleValidationErrors, async (req, res)
=> {
    // ... validation code ...
    const { emoji } = req.body;
    const validEmojis = ["👍", "😂", "😯", "🔥", "🙏"];
    if (!validEmojis.includes(emoji)) {
        return res.status(400).json({ error: 'Invalid emoji' });
    }
    // ...
});
```

Check	Status	Details
Type Validation	complete	MongoDB ID format validated
Length Validation	N/A	Single emoji character
Format Validation	complete	Allowlist validation for emojis
Allowlist Used	complete	**EXCELLENT** - Explicit emoji allowlist
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	complete	Mongoose ORM used

Status: complete **EXCELLENT IMPLEMENTATION** (Allowlist pattern)

Entry Point 7: Search Posts (`/api/search-posts`)

```
app.get('/api/search-posts', query("query").trim().notEmpty().withMessage("Search
query required"), handleValidationErrors, async (req, res) => {
    const { query } = req.query;
    // ...
    const posts = await Post.find({
        community: { $in: user.joinedCommunities },
        $or: [
            { title: { $regex: query, $options: 'i' } },
            { brief: { $regex: query, $options: 'i' } }
        ]
    });
});
```

Check	Status	Details
Type Validation	complete	String validation
Length Validation	not implemented	No max length on search query
Format Validation	not implemented	No pattern validation

Allowlist Used	not implemented	Free-text search
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	partial	Regex used but with user input

partial SECURITY CONCERN: While Mongoose handles parameterization, the regex could be optimized:

- Add length limit: `.isLength({ max: 100 })`
- Escape regex special characters to prevent ReDoS

Recommendation:

```
const escapedQuery = query.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
```

Entry Point 8: Get Post by ID (`/api/post/:id`)

```
app.get('/api/post/:id', validateId, handleValidationErrors, async (req, res) => {
    const post = await Post.findById(req.params.id);
    // ...
});

const validateId = [
    param("id").isMongoId().withMessage("Invalid post ID")
];
```

Check	Status	Details
Type Validation	complete	MongoDB ID format validated
Length Validation	N/A	Fixed format (24 char hex)
Format Validation	complete	`isMongoId()` validates format
Allowlist Used	N/A	ID format validation
Sanitization	complete	`filterXSS` applied globally
Parameterized Query	complete	Mongoose `findById()` used

Status: complete **WELL IMPLEMENTED**

A.iii) Input Sanitization Summary

Global Sanitization Middleware

```
const sanitizeInput = (req, res, next) => {
    if (req.body) {
        Object.keys(req.body).forEach(key => {
            if (typeof req.body[key] === "string") {
                req.body[key] = filterXSS(req.body[key]);
```

```
            }
        });
    }
    if (req.query) {
        Object.keys(req.query).forEach(key => {
            if (typeof req.query[key] === "string") {
                req.query[key] = filterXSS(req.query[key]);
            }
        });
    }
    if (req.params) {
        Object.keys(req.params).forEach(key => {
            if (typeof req.params[key] === "string") {
                req.params[key] = filterXSS(req.params[key]);
            }
        });
    }
    next();
};

app.use(sanitizeInput);
```

Aspect	Status	Details
Coverage	complete	Body, Query, Params
Library	complete	`xss` (filterXSS) - industry standard
Position	complete	Applied early in middleware chain
Type Check	complete	Only processes string types

A.iv) Database Query Security

Mongoose ORM Usage

All database queries use Mongoose ORM which provides:

- **Automatic parameterization** - No string concatenation
- **Type casting** - Input converted to expected types
- **Schema validation** - Data validated against schema

```
// complete SECURE — Mongoose handles parameterization
const user = await User.findOne({ username: req.body.username });
const post = await Post.findById(req.params.id);
const posts = await Post.find({ community: { $in: user.joinedCommunities } });
```

Query Type	Method	Parameterized
Find One	`User.findOne()`	complete
Find By ID	`Post.findById()`	complete

Find In Array	`$in` operator	complete
Regex Search	`$regex`	partial (escaped input recommended)
Insert	`new User().save()`	complete
Update	`user.save()`	complete

B) Output Encoding

B.i) Output Encoding Analysis

Server-Side Output

Location	Method	Encoding Applied
JSON Responses	`res.json()`	complete Automatic JSON encoding
Redirects	`res.redirect()`	complete URL encoding handled
Static Files	`res.sendFile()`	complete No user data

Client-Side Output (HTML/JavaScript)

Vue.js Templates (post-view.html)

```
// Vue.js auto-escapes interpolations by default
<div class="post-header">
    <span class="community-name">{{ post.community }}</span>
    <span class="post-meta">Posted by {{ post.username }}</span>
</div>
<h3 class="post-title">{{ post.title }}</h3>
<p class="post-brief">{{ post.brief }}</p>
<div class="comment-content">{{ comment.content }}</p>
```

Aspect	Status	Details
Framework	complete	Vue.js 3
Auto-Escaping	complete	`{{ }}` interpolations are HTML-encoded
Context	complete	HTML entity encoding applied
v-html Usage	not implemented	Not used (good - would bypass escaping)

Status: complete **SECURE** - Vue.js provides automatic context-aware HTML encoding

---

Vanilla JavaScript Templates (home.html)

```
// partial POTENTIAL XSS RISK - Using innerHTML with template literals
container.innerHTML = posts.map(post => `
```

```
    <div class="post" data-id="${post._id}" onclick="viewPost(event,
'${post._id}')">
        <div class="post-header">
            <span class="community-name">${post.community}</span>
            <span class="post-meta">Posted by ${post.username}</span>
        </div>
        <h3 class="post-title">${post.title}</h3>
        <p class="post-brief">${post.brief}</p>
        // ...
    </div>
`).join('');
```

Aspect	Status	Details
Method	partial	`innerHTML` with template literals
Encoding	not implemented	No explicit encoding applied
Risk	partial	XSS possible if server sanitization fails
Mitigation	complete	Server-side `filterXSS` provides protection

partial SECURITY CONCERN: While server-side sanitization helps, defense-in-depth recommends client-side encoding too.

Current Protection:

- Server-side `filterXSS` sanitizes before storage
- CSP headers limit script execution

Recommended Improvement:

```
// Create text element to safely encode
function encodeHTML(str) {
    const div = document.createElement('div');
    div.textContent = str;
    return div.innerHTML;
}

// Use in templates
<span class="community-name">${encodeHTML(post.community)}</span>
```

Dynamic Content Insertion

```
// partial Using innerHTML with user-controlled community names
list.innerHTML = communities.map(c => `
    <li>
        <div class="community">
            <span>${c}</span>  // Community name from database
            <button class="join-btn" onclick="leaveCommunity('${c}')">Leave</button>
        </div>
```

```
    </li>
`).join("");
```

Aspect	Status	Details
Method	partial	`innerHTML` with template literals
Event Handler	partial	Inline `onclick` with string interpolation
Encoding	not implemented	No explicit encoding
Mitigation	complete	Server-side sanitization + CSP

partial SECURITY CONCERN: Inline event handlers with string interpolation could be exploited if sanitization is bypassed.

Recommended Improvement:

```
// Use event listeners instead of inline handlers
communities.forEach(c => {
    const li = document.createElement('li');
    const div = document.createElement('div');
    div.className = 'community';

    const span = document.createElement('span');
    span.textContent = c;  // complete Safe text insertion

    const btn = document.createElement('button');
    btn.className = 'join-btn';
    btn.textContent = 'Leave';
    btn.addEventListener('click', () => leaveCommunity(c));

    div.appendChild(span);
    div.appendChild(btn);
    li.appendChild(div);
    list.appendChild(li);
});
```

B.ii) Output Encoding Context Summary

Context	Encoding Method	Status
HTML Body (Vue.js)	Auto HTML entity encoding	complete Secure
HTML Body (Vanilla JS)	None (relies on server)	partial Needs improvement
JavaScript Strings	None	partial Needs improvement
URL Parameters	`encodeURIComponent()`	complete Used in search
JSON Response	`res.json()` auto-encoding	complete Secure
CSS Context	N/A	No user data in CSS

HTML Attributes	None explicit	partial Needs improvement

C) Logging & Monitoring

C.i) Important Events Logging

Authentication Events

```
// Login - SUCCESS not logged, FAILURE logged
app.post("/login", loginLimiter, validateLogin, handleValidationErrors, async (req,
res) => {
    try {
        const user = await User.findOne({ username: req.body.username });

        if (!user) {
            return res.status(401).json({ error: "Invalid username or password" });
        }

        const validPassword = await bcrypt.compare(req.body.password,
user.password);
        if (!validPassword) {
            return res.status(401).json({ error: "Invalid username or password" });
        }

        req.session.username = user.username;
        res.redirect("/home.html");
    } catch (err) {
        console.error("Login error:", err);  // complete Error logged
        res.status(500).json({ error: "An error occurred" });
    }
});
```

Event	Logged	Details	Timestamp	Sensitive Data
Login Success	not implemented	No log entry	N/A	N/A
Login Failure (user not found)	not implemented	No log entry	N/A	N/A
Login Failure (wrong password)	not implemented	No log entry	N/A	N/A
Login Error (exception)	complete	`console.error("Login error:", err)`	complete Implicit	partial May contain error details

Registration Success	not implemented	No log entry	N/A	N/A
Registration Failure	complete	`console.error("Registration error:", err)`	complete Implicit	partial May contain error details

---

Session Events

```
app.get("/logout", (req, res) => {
    req.session.destroy();
    res.redirect("/login.html");
});  // not implemented No logout logging
```

Event	Logged	Details
Session Created	not implemented	No log on login success
Session Destroyed	not implemented	No log on logout
Session Expired	not implemented	No log

Error Events

```
// Global error handler
app.use((err, req, res, next) => {
    console.error("Unhandled error:", err);  // complete All errors logged
    // ...
});

// Endpoint-specific error handling
} catch (error) {
    console.error("Error fetching posts:", error);  // complete Errors logged
    res.status(500).json([]);
}
```

Event	Logged	Details
Unhandled Errors	complete	`console.error("Unhandled error:", err)`
Database Errors	complete	`console.error("Error fetching posts:", error)`
Validation Errors	not implemented	Not explicitly logged (handled by middleware)
404 Errors	not implemented	Not logged

Authorization Events

```
// Session check without logging
if (!req.session.username) {
    return res.status(401).json({ error: "Session expired. Please log in again." });
}  // not implemented No log for unauthorized access attempts
```

Event	Logged	Details
Unauthorized Access	not implemented	No log
Forbidden Access	not implemented	No log
Permission Denied	not implemented	No log

C.ii) Log Timestamp Analysis

```
console.error("Login error:", err);
console.error("Error fetching posts:", error);
```

Aspect	Status	Details
Timestamp Present	partial	Implicit via `console.error()`
Timestamp Format	partial	Default Node.js format (not standardized)
Timezone	partial	System default (not UTC)
Log Level	partial	Only `error` used (no info/warn/debug)

Current Output Format:

```
Error fetching posts: Error: Connection timeout
    at /path/to/server.js:123:45
```

Recommended Format:

```
const log = (level, message, meta = {}) => {
    const entry = {
        timestamp: new Date().toISOString(),  // complete ISO 8601 UTC
        level,                                // complete Explicit level
        message,
        ...meta                               // complete Structured data
    };
    console[level](JSON.stringify(entry));
};

// Usage
log('error', 'Login failed', {
    username: req.body.username,  // partial Don't log passwords!
    ip: req.ip,
```

```
    reason: 'invalid_password'
});
```

C.iii) Sensitive Data in Logs

Current Logging Practices

```
// partial POTENTIAL ISSUE - Full error object logged
console.error("Login error:", err);
console.error("Registration error:", err);

// complete GOOD - Specific data logged
console.error("Error fetching posts:", error);
```

Data Type	Logged	Risk
Passwords	partial	May be in error stack traces
Email Addresses	partial	May be in error details
Session IDs	not implemented	Not logged
IP Addresses	not implemented	Not logged (could be useful for security)
Usernames	partial	May be in error details
Full Request Body	partial	May be in error object

Sensitive Data Exposure Risk

```
// Risk: Error object may contain sensitive data
console.error("Login error:", err);

// If err contains request body:
{
    message: "Invalid username or password",
    body: {
        username: "user@example.com",
        password: "secretpassword123"  // partial COULD BE LOGGED!
    }
}
```

Recommendation:

```
// Log only safe error properties
console.error("Login error:", {
    message: err.message,
    stack: err.stack,
```

```
    // Never log: err.body, err.password, err.token
});
```

Findings Summary

Critical Findings

ID	Finding	Risk	Recommendation
F1	No output encoding in vanilla JS templates	Medium	Use DOM methods or encoding function
F2	Inline event handlers with string interpolation	Medium	Use addEventListener instead
F3	Authentication events not logged	Medium	Add structured logging for auth events
F4	No max length on search query	Low	Add `.isLength({ max: 100 })`
F5	Regex special characters not escaped	Low	Escape before regex use
F6	Sensitive data may be in error logs	Medium	Sanitize error objects before logging

---

Recommendations Priority

High Priority

1. **Add client-side output encoding** for vanilla JavaScript templates
2. **Replace inline event handlers** with `addEventListener`
3. **Implement structured logging** with explicit timestamps (UTC)
4. **Log authentication events** (success/failure) without sensitive data

Medium Priority

5. **Add length validation** to search query endpoint
6. **Escape regex special characters** in search functionality
7. **Add community name pattern validation** in create-community
8. **Sanitize error objects** before logging to prevent sensitive data exposure

Low Priority

9. **Add logging for 404 errors** (rate limiting detection)
10. **Add log levels** (info, warn, error, debug)
11. **Consider logging IP addresses** for security monitoring (with privacy compliance)

Compliance Checklist

Requirement	Status	Evidence
Server-side validation on all inputs	complete	express-validator on all endpoints
Type validation	complete	express-validator type checks

Length validation	partial	Missing on login, search
Format validation	partial	Missing on some fields
Allowlist validation	complete	Emoji allowlist, username pattern
Input sanitization	complete	filterXSS global middleware
Parameterized queries	complete	Mongoose ORM used
HTML output encoding	partial	Vue.js yes, vanilla JS no
JavaScript output encoding	not implemented	Not implemented
URL encoding	complete	encodeURIComponent used
Authentication logging	not implemented	Not implemented
Error logging	complete	console.error used
Timestamps in logs	partial	Implicit, not standardized
Sensitive data excluded from logs	partial	Not guaranteed

Task 2

Semgrep Analysis Results

Figure: Semgrep static analysis scan results showing identified security issues and warnings.

Conclusion

The Quotient application has a **solid security foundation** with:

- Comprehensive server-side validation using express-validator
- Global XSS sanitization middleware
- Mongoose ORM preventing SQL/NoSQL injection
- Vue.js auto-escaping for most templates

Key improvements needed:

1. Client-side output encoding for vanilla JavaScript
2. Structured logging with authentication event tracking
3. Ensure sensitive data is never logged

Overall Security Posture: partial **MODERATE** - Good foundation with specific improvements needed